

Chapter 04. 딥러닝 시작

- 일시: 2026.04.07.
- 발표자: 좌은경

CONTENTS

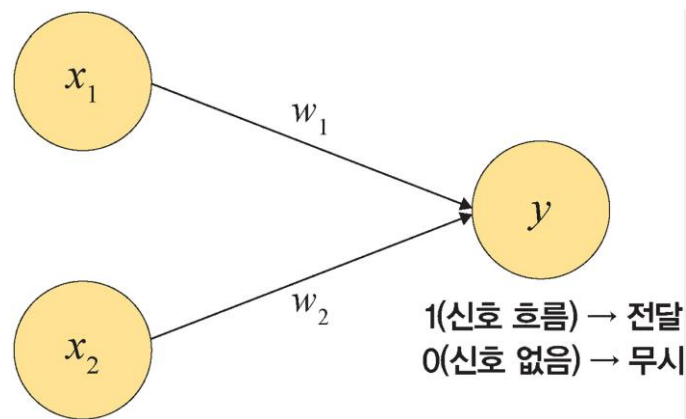


1. 인공신경망의 한계와 딥러닝 출현

2. 딥러닝 구조

1. 인공 신경망의 한계와 딥러닝 출현 - 퍼셉트론

- 퍼셉트론(Perceptron)
 - 오늘날 신경망(딥러닝)의 기원이 되는 알고리즘(초기 인공신경망)
 - 다수의 신호를 받아 하나의 신호로 출력함
 - 이 신호를 입력으로 받아 '흐른다(1)/안 흐른다(0)' 는 정보를 앞으로 전달하는 원리



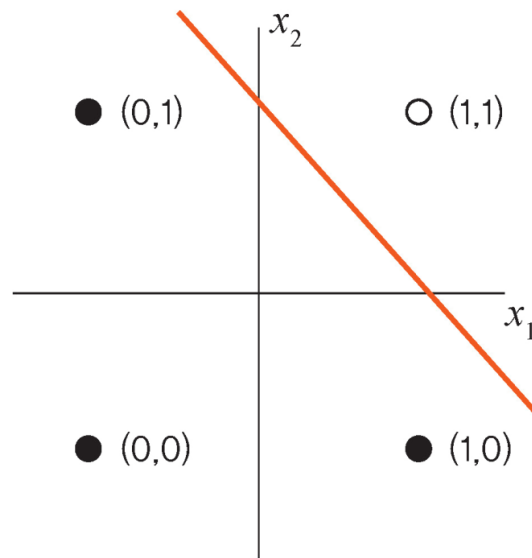
x_1, x_2 : 입력 w_1, w_2 : 가중치 y : 출력

▲ 퍼셉트론의 원리

1. 인공 신경망의 한계와 딥러닝 출현 - 컴퓨터가 논리적으로 인식하는 방법

- AND 게이트 : 모든 입력이 1일 때 작동, 입력 중 어떤 하나도 0이면 멈춤

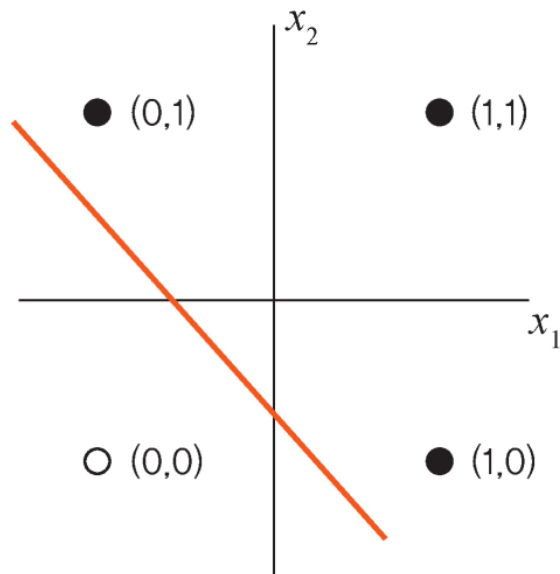
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



1. 인공 신경망의 한계와 딥러닝 출현 - 컴퓨터가 논리적으로 인식하는 방법

- OR 게이트 : 입력에서 둘 중 하나만 1이거나, 둘 다 1일 때 작동
→ 입력 모두가 0일 경우만 0

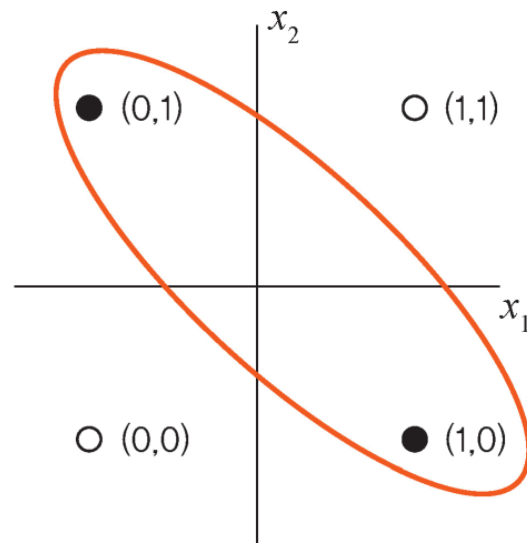
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



1. 인공 신경망의 한계와 딥러닝 출현 - 컴퓨터가 논리적으로 인식하는 방법

- XOR 게이트 : 둘 중 하나만 '1'일 때 작동

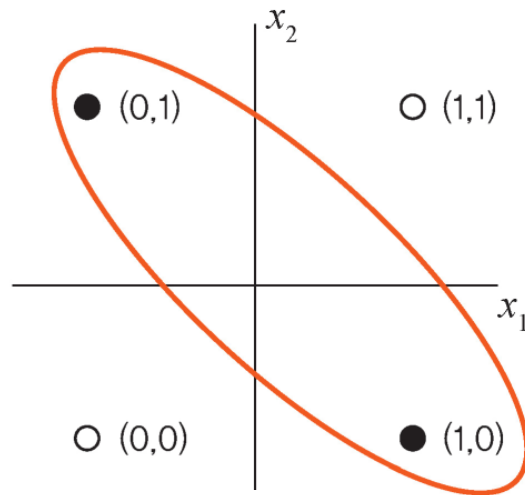
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



1. 인공 신경망의 한계와 딥러닝 출현 - 컴퓨터가 논리적으로 인식하는 방법

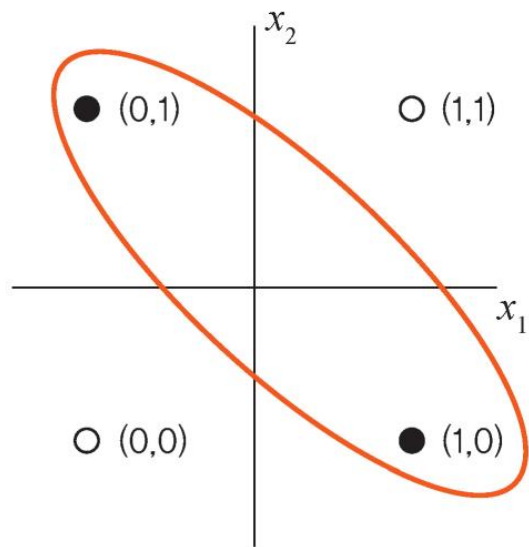
- XOR 게이트 : 둘 중 하나만 '1'일 때 작동

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



데이터가 비 선형적으로 분리됨 → 제대로 된 분류가 어렵다
단층 퍼셉트론에서는 AND, OR 연산 학습 가능 / XOR 학습 불가

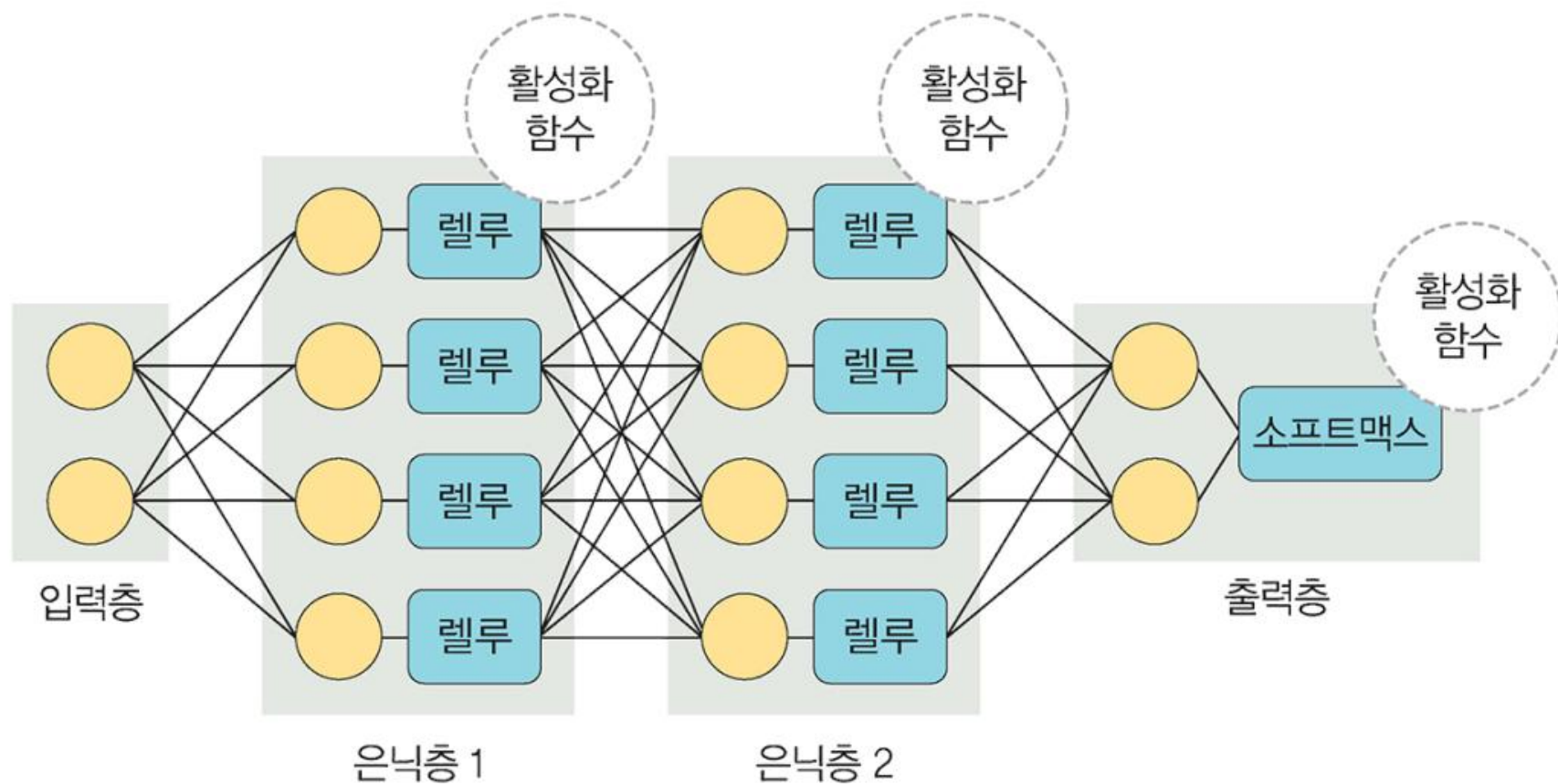
1. 인공 신경망의 한계와 딥러닝 출현 - 컴퓨터가 논리적으로 인식하는 방법



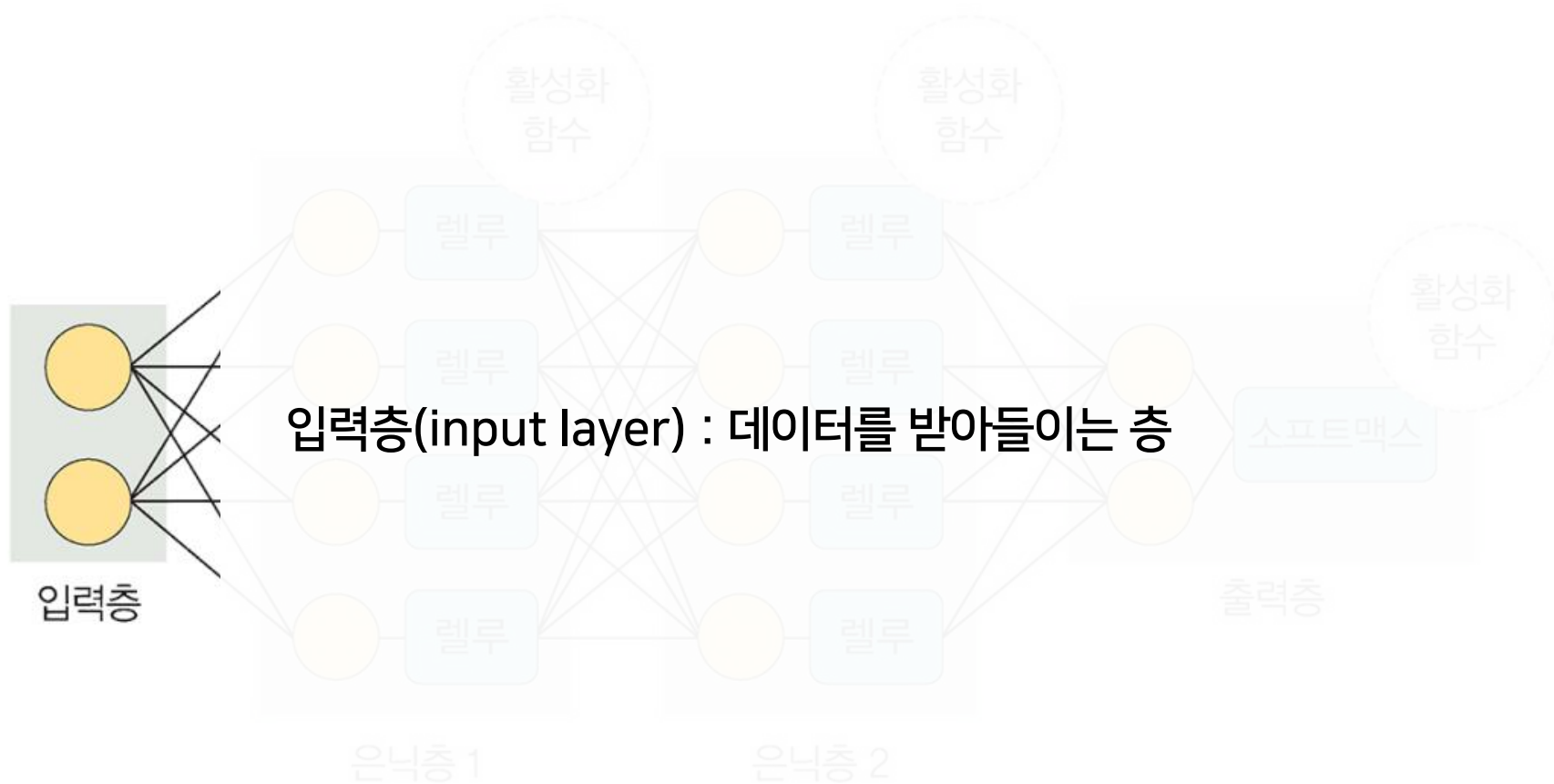
단층 퍼셉트론 → XOR 해결불가(비선형 문제)

- 해결방안 : 다층 퍼셉트론(multi-layer perceptron)
 - 입력층, 출력층 사이에 하나이상의 중간층(은닉층) 추가
 - 비선형 데이터 학습 가능
- 심층 신경망(Deep Neural Network, DNN)
 - 입력층, 출력층 사이에 은닉층이 여러 개 있는 신경망

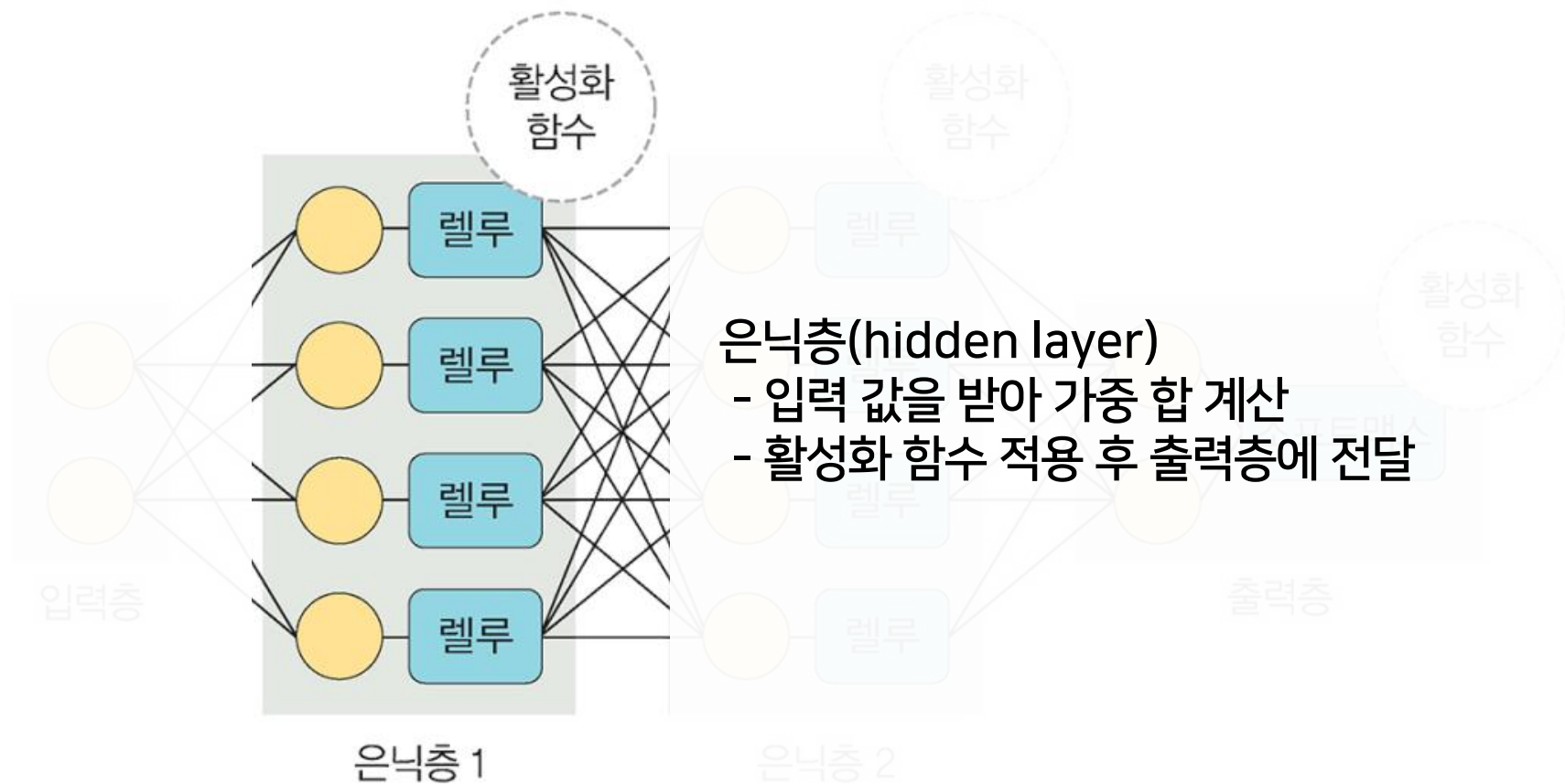
2. 딥러닝 구조 - 딥러닝 용어



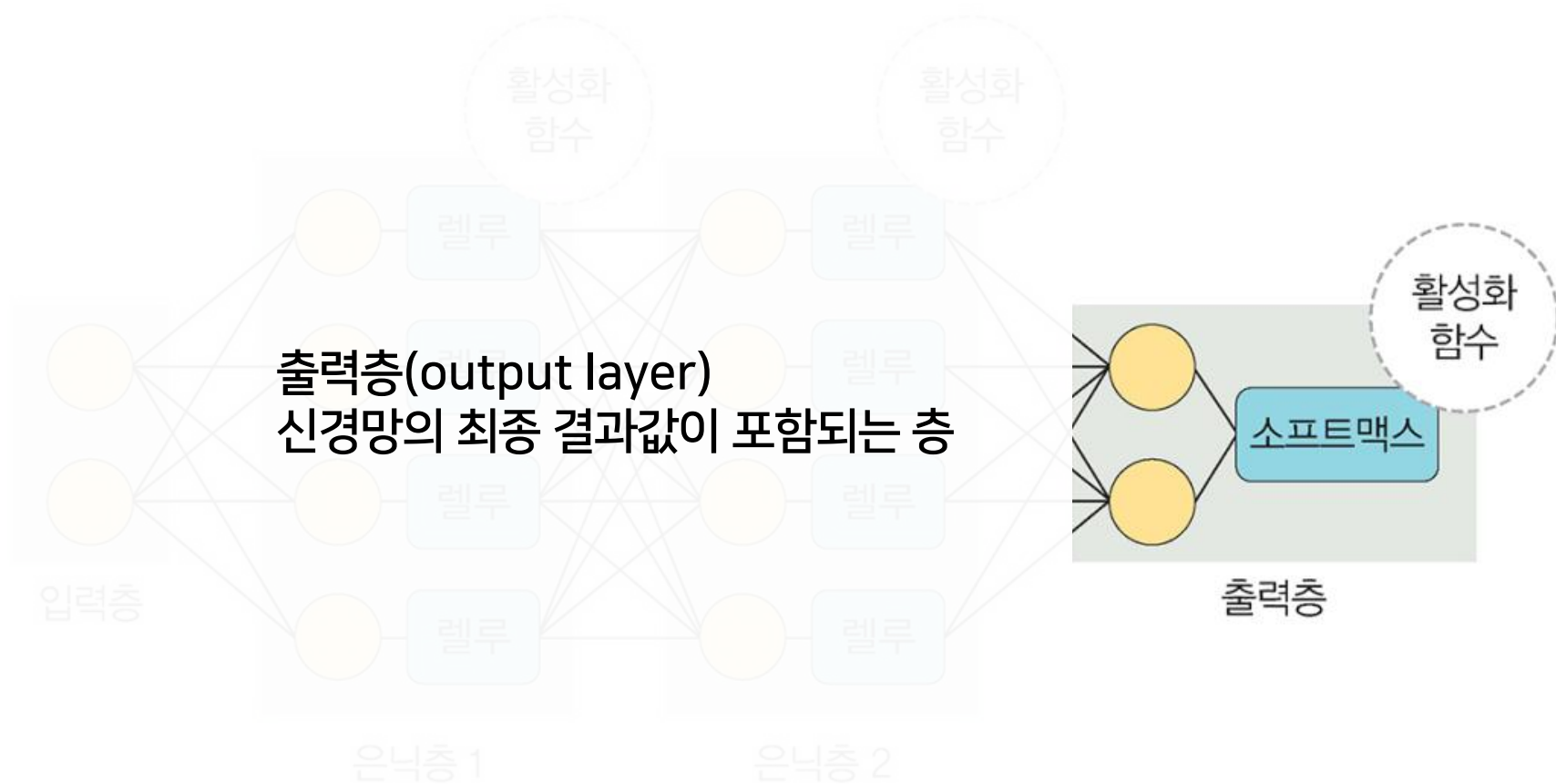
2. 딥러닝 구조 - 딥러닝 용어



2. 딥러닝 구조 - 딥러닝 용어



2. 딥러닝 구조 - 딥러닝 용어

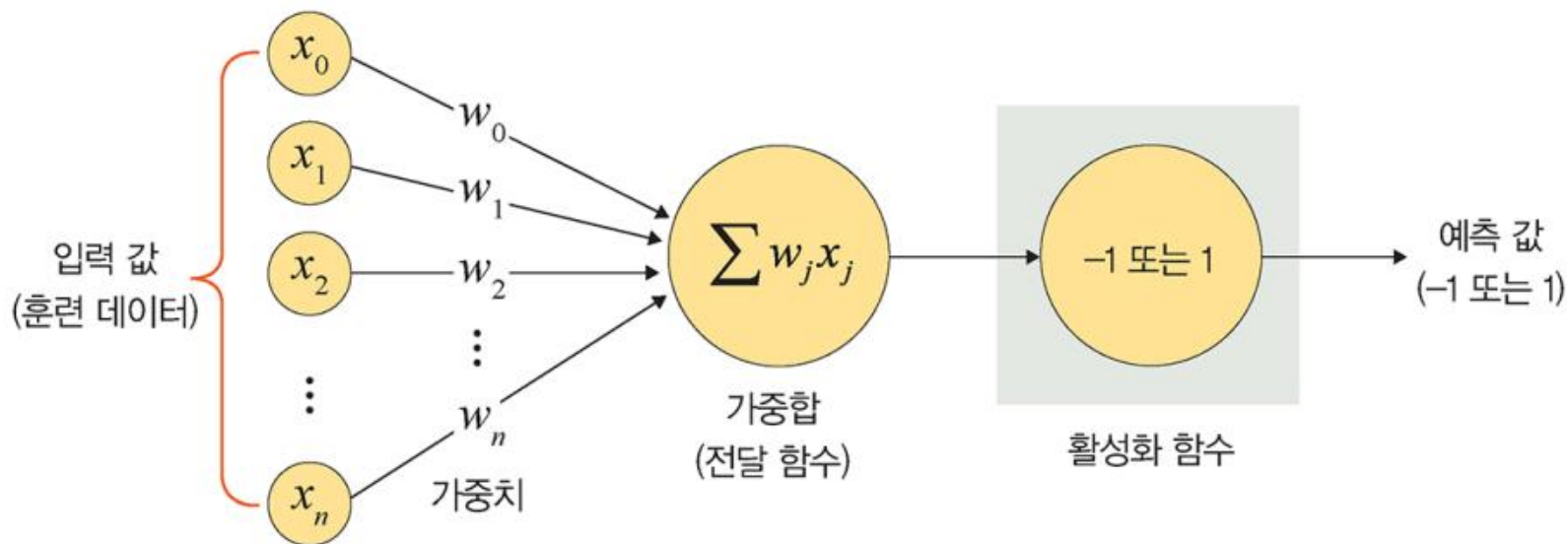


2. 딥러닝 구조 - 딥러닝 용어

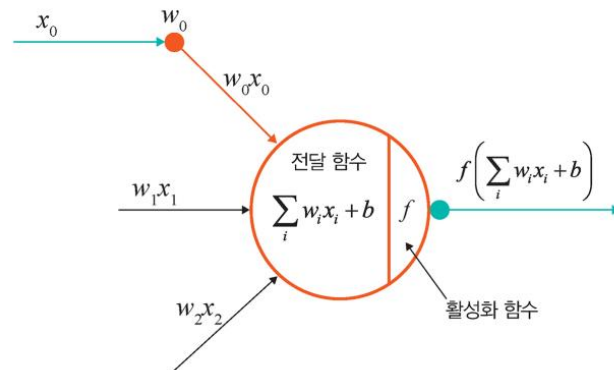
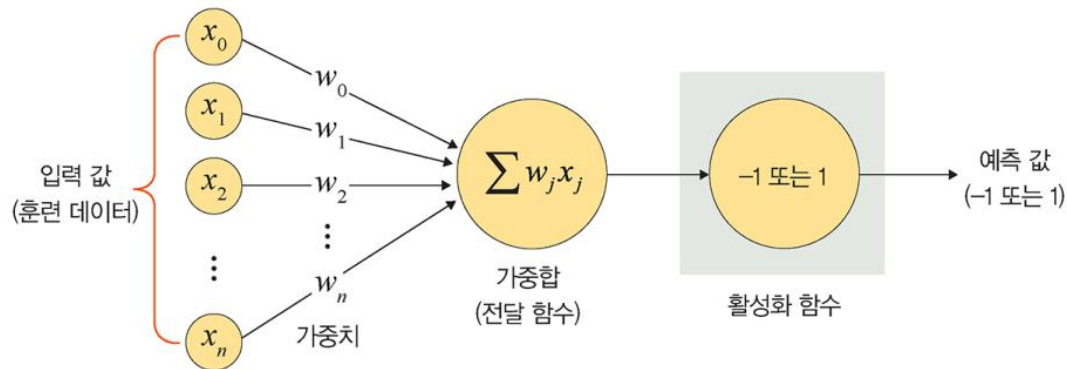
구분	구성요소	설명
	가중치(weight)	노드와 노드 간 연결 강도
	바이어스(bias)	가중합에 더해주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할
	가중합(weighter sum), 전달함수	가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

2. 딥러닝 구조 - 가중치

- 가중치(weight) : 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소



2. 딥러닝 구조 - 가중합 또는 전달함수

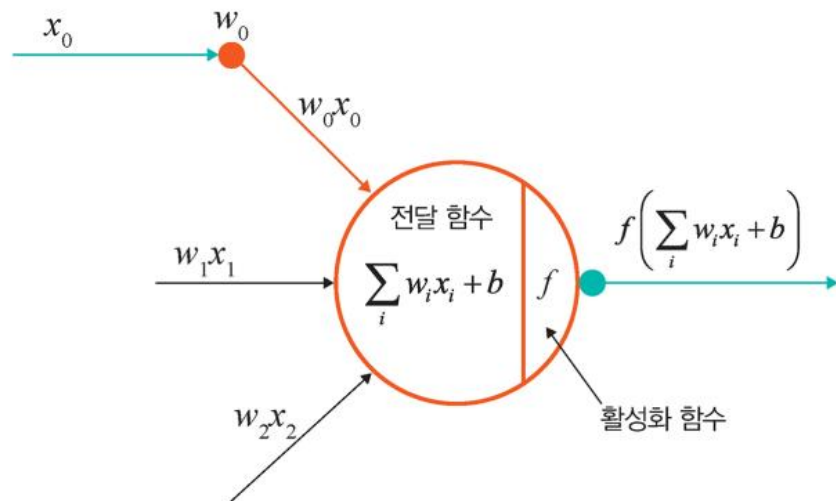


- 가중합(Weighted Sum)
 - 입력값 x 가중치 + 바이어스
 - 활성화 함수로 전달

$$\sum_i w_i x_i + b$$

(w : 가중치, b : 바이어스)

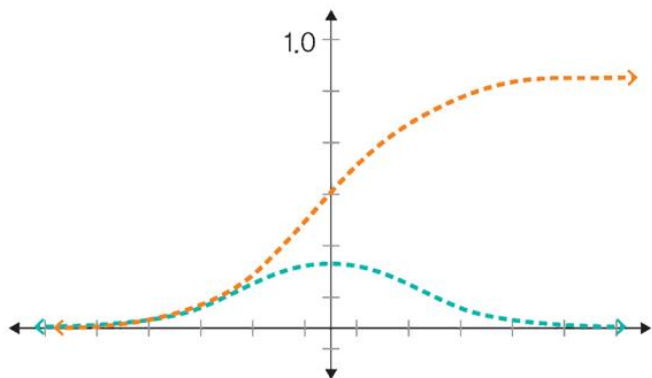
2. 딥러닝 구조 - 활성화 함수



- 활성화 함수
 - 가중 합 결과를 입력으로 받음
 - 비선형 변환을 통해 출력 결정
 - 예 : Sigmoid, Tanh, ReLU

2. 딥러닝 구조 - 활성화 함수(시그모이드 함수)

- 시그모이드 함수
 - 선형함수의 결과를 0~1 사이에서 비선형 형태로 변환
 - 분류문제(로지스틱 회귀) 확률적으로 표현하는데 사용
- 단점 : 기울기 소멸 문제 발생 → 딥러닝에서 잘 사용X
 - ※ 딥러닝 문제와 해결방안에서 다룰 예정



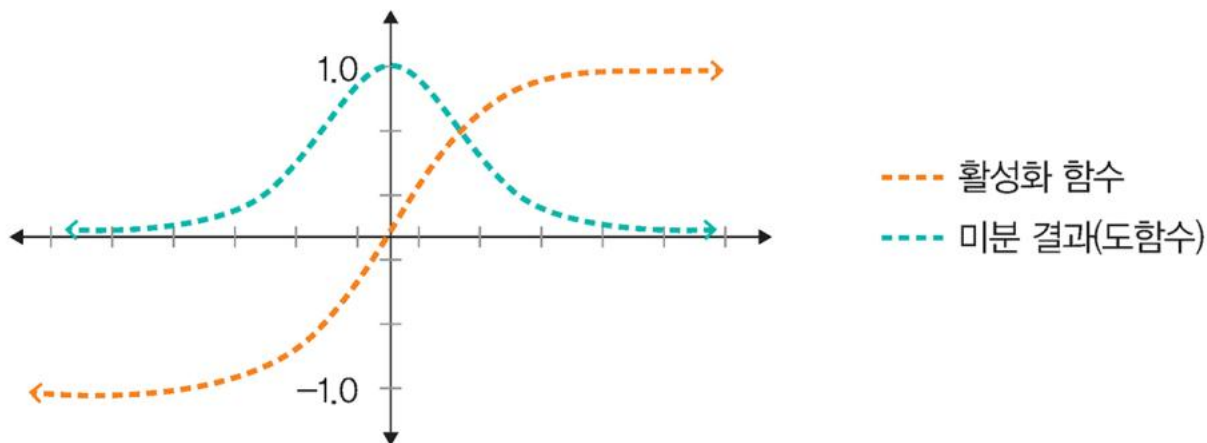
$$f(x) = \frac{1}{1 + e^{-x}}$$

--- 활성화 함수
--- 미분 결과(도함수)

- 출력값 : 0~1 범위

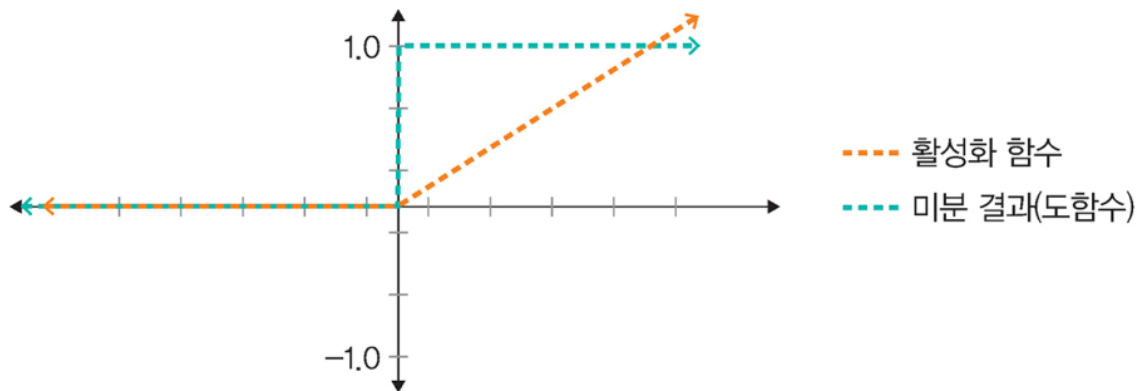
2. 딥러닝 구조 - 활성화 함수(하이퍼볼릭 탄젠트 함수)

- 하이퍼볼릭 탄젠트 함수
 - 선형함수의 결과 $-1 \sim 1$ 사이에서 비선형 형태로 변환
 - 시그모이드의 결과값의 평균이 양수로 편향된 문제 해결위해 사용
 - 단점 : 기울기 소멸 문제는 여전히 발생함



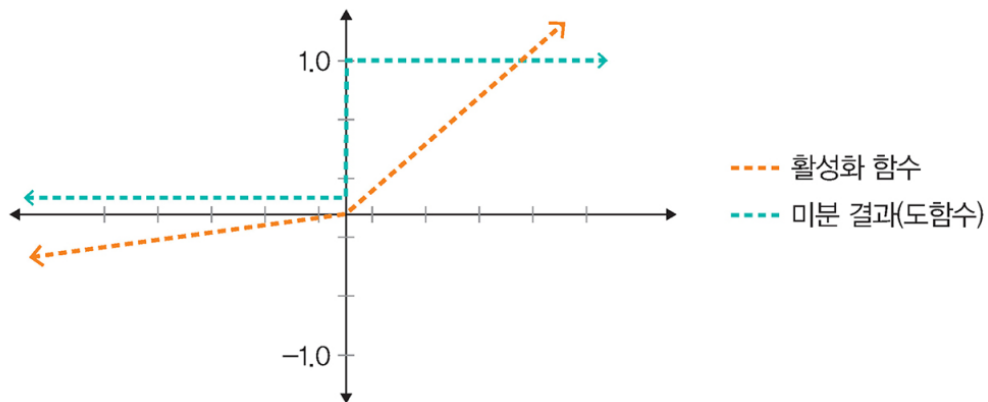
2. 딥러닝 구조 - 활성화 함수(렐루함수)

- 렐루(ReLU) 함수
 - 음수일때 0출력, 양수일때 입력값 그대로 출력
 - 장점 : 계산이 단순하여 학습속도가 빠름
기울기 소멸 문제 발생 X
 - 단점 : 음수일때 0출력 → 학습 안됨
 - 주로 은닉층에 사용



2. 딥러닝 구조 - 활성화 함수(리키 렐루함수)

- 리키렐루(Leaky ReLU) 함수
 - 음수일때 0이아닌 0.001처럼 매우 작은수 출력
 - 렐루함수를 사용할 때 생기는 문제 해결위해 사용



2. 딥러닝 구조 - 활성화 함수(소프트맥스 함수)

- 소프트맥스(softmax)함수
 - 출력노드의 활성화 함수로 사용됨
 - 0~1 사이로 출력되도록 정규화 하여 출력 값들의 총합이 항상 1

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

2. 딥러닝 구조 - 손실함수

- 손실함수(Loss Function)
 - 예측값과 실제값의 차이를 측정하는 함수
 - 값이 0에 가까울 수록 성능이 좋음
- 대표 예시
 - MSE(평균 제곱 오차) : 회귀 문제
 - Cross Entropy(크로스 엔트로피) : 분류 문제

2. 딥러닝 구조 - 손실함수(평균 제곱 오차)

- 평균제곱오차
 - 실제 값과 예측 값 차이를 제곱하여 평균을 낸 것
 - 실제, 예측 값의 차이가 클수록 평균 제곱 오차의 값도 커짐
 - 값이 작을수록 예측력이 좋다는것을 의미함
 - 회귀에서 손실함수로 주로 사용됨

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

2. 딥러닝 구조 - 손실함수(크로스 엔트로피 오차)

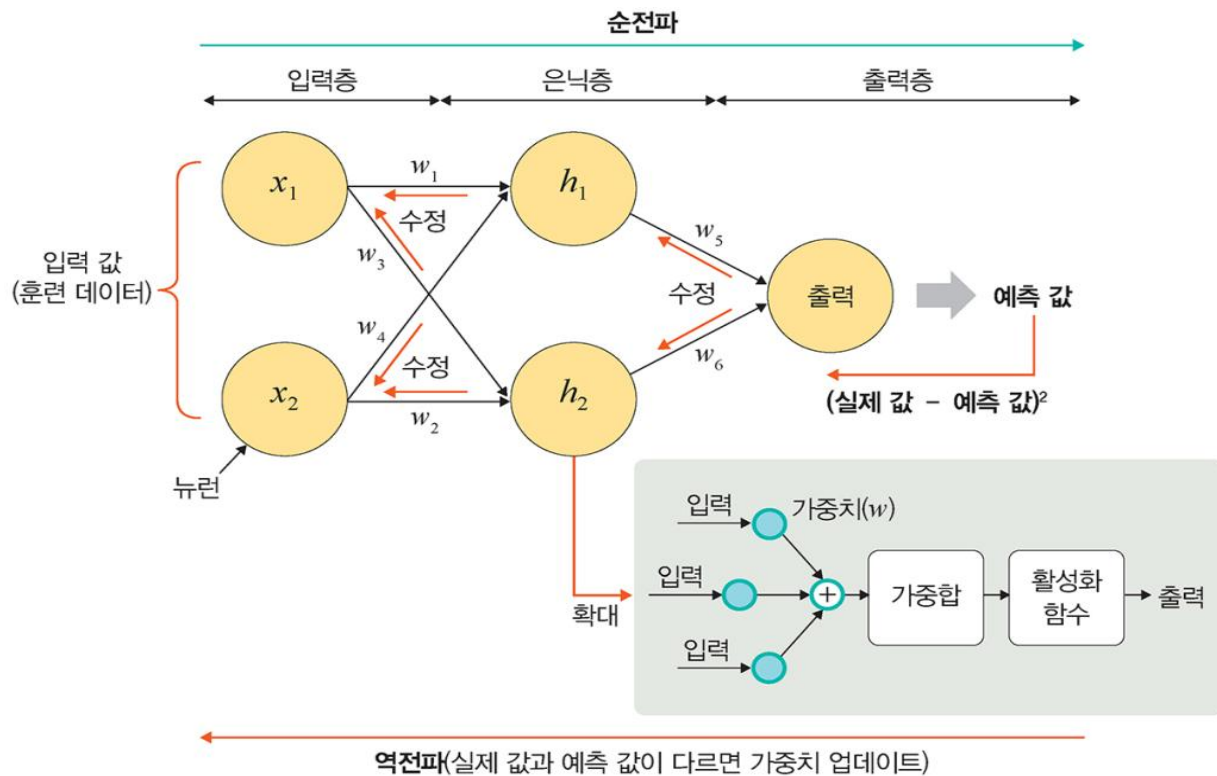
- 크로스 엔트로피 오차
 - 예측 확률과 실제값의 차이를 측정하는 손실 함수
 - 분류 문제에서 사용
 - 정답일수록 손실 값이 작아짐
 - 틀릴수록 손실 값이 크게 증가

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

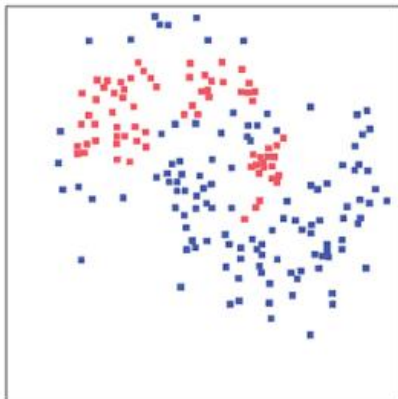
2. 딥러닝 구조 - 딥러닝 학습(순전파, 역전파)

- 딥러닝 학습은 순전파 역전파 두 단계로 진행

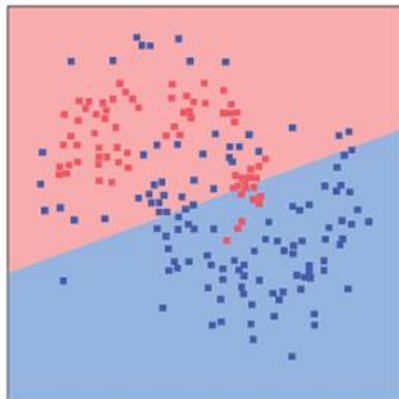


2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안

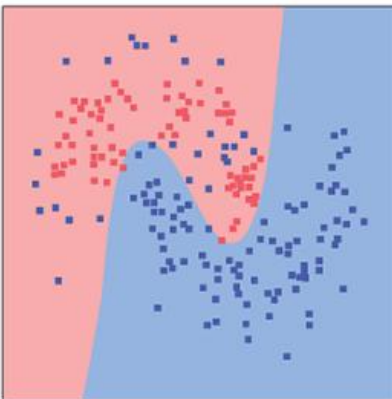
- 딥러닝의 핵심 : 여러 은닉층을 결합하여 비선형 영역을 표현 하는 것



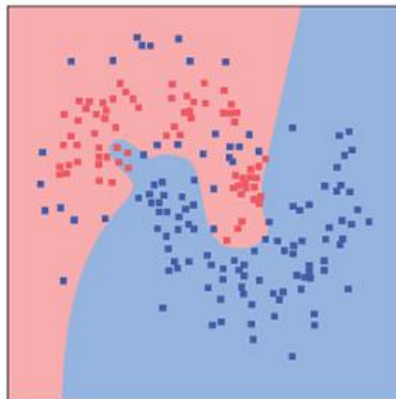
데이터



로지스틱 회귀



은닉층 세 개

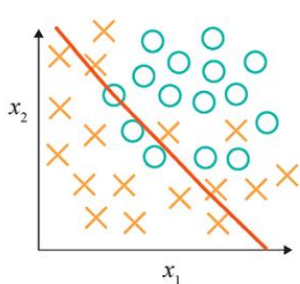


은닉층 50개

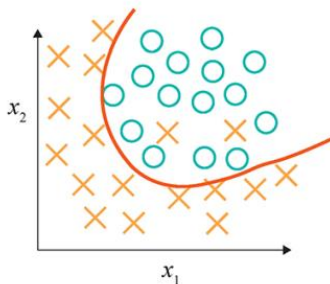
2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(과적합 문제 발생)

- 과적합 문제(over-fitting)

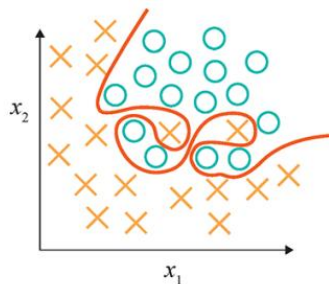
- 훈련 데이터를 과하게 학습해서 발생함
- 훈련데이터 : 오차감소 / 실제(검증)데이터 : 오차 증가



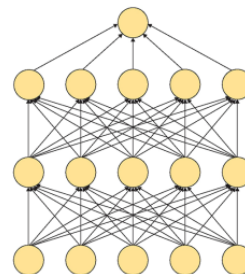
과소적합
(under-fitting)



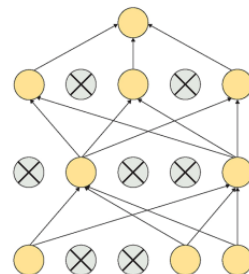
적정적합
(generalized-fitting)



과적합
(over-fitting)



일반적인 신경망



드롭아웃이 적용된 신경망

과적합 해결방법 : 드롭아웃(drop out)

- 학습과정 중 임의로 일부 노드를 학습에서 제외시킴

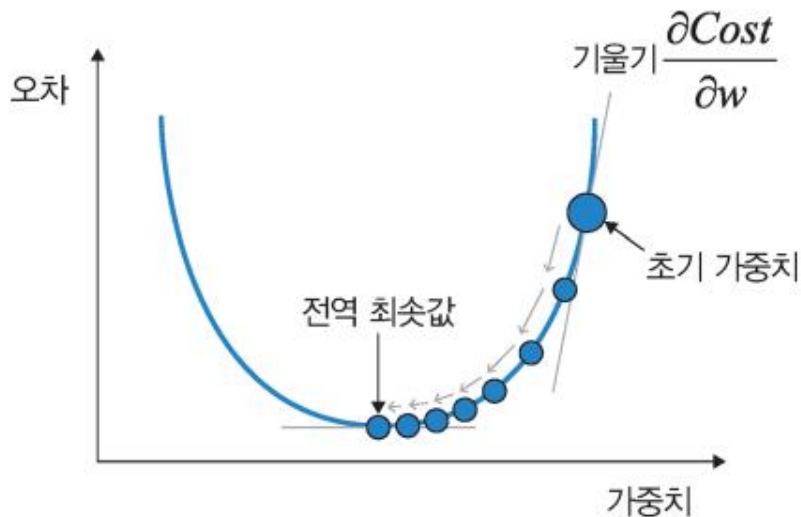
2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(기울기 소멸 문제)

- 기울기 소멸 문제
 - 은닉층이 많은 신경망에서 주로 발생
 - 출력층에서 은닉층으로 전달되는 오차가 크게 줄어 학습이 되지 않는 현상
 - 기울기 가 소멸되기 때문에 학습되는 양이 '0'에 가까워져서 학습이 더디게 진행
오차를 더 줄이지 못하고 그 상태로 수렴하는 현상

기울기 소멸 해결방법 : 렐루 활성화 함수 사용하면 해결 가능

2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(성능이 나빠지는 문제 발생)

- 경사하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때 까지 기울기가 낮은 쪽으로 계속 이동시키는 과정 반복 → 성능이 나빠지는 문제 발생



2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(성능이 나빠지는 문제 발생)

배치 경사 하강법



$$W = W - a \nabla J(W, b)$$

확률적 경사 하강법



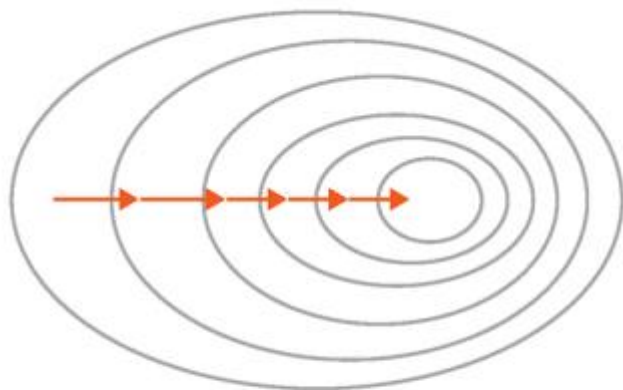
$$W = W - a \nabla J(W, b, x^{(z)}, y^{(z)})$$

미니 배치 경사 하강법

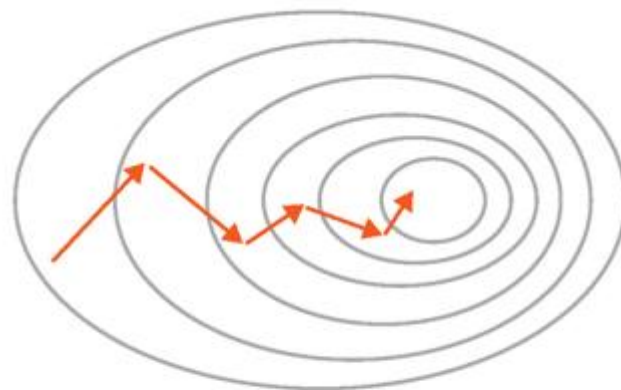


$$W = W - a \nabla J(W, b, x^{(z:z+bs)}, y^{(z:z+bs)})$$

2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(성능이 나빠지는 문제 발생)

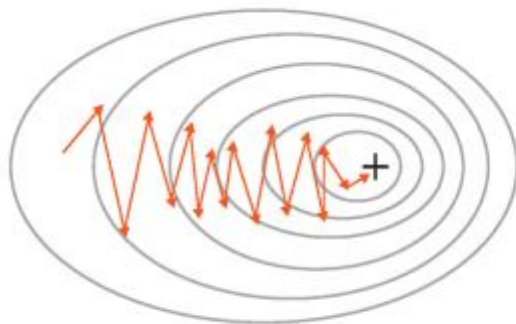
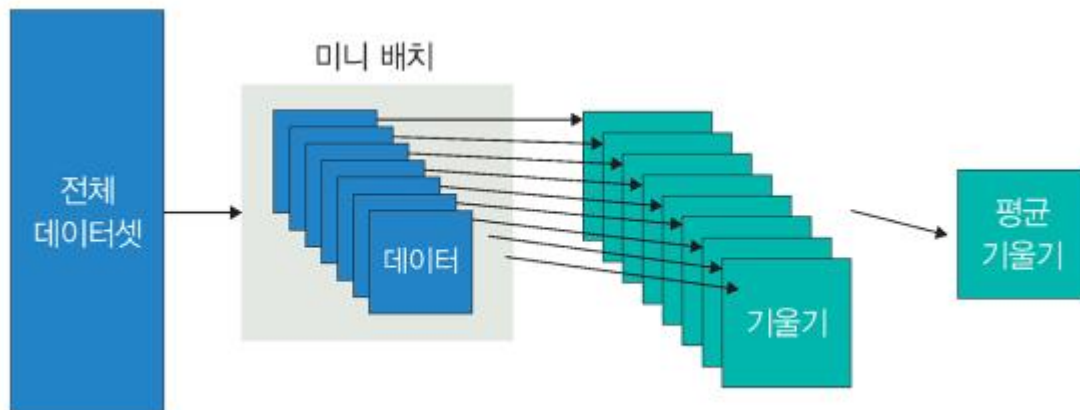


배치 경사 하강법

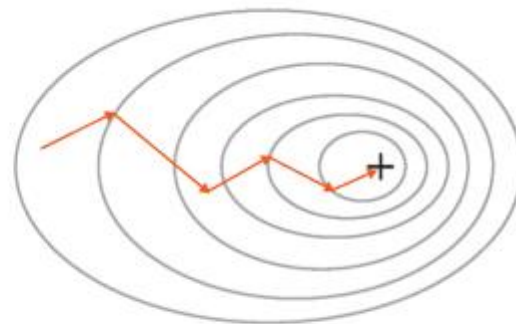


확률적 경사 하강법

2. 딥러닝 구조 - 딥러닝의 문제점과 해결 방안(성능이 나빠지는 문제 발생)



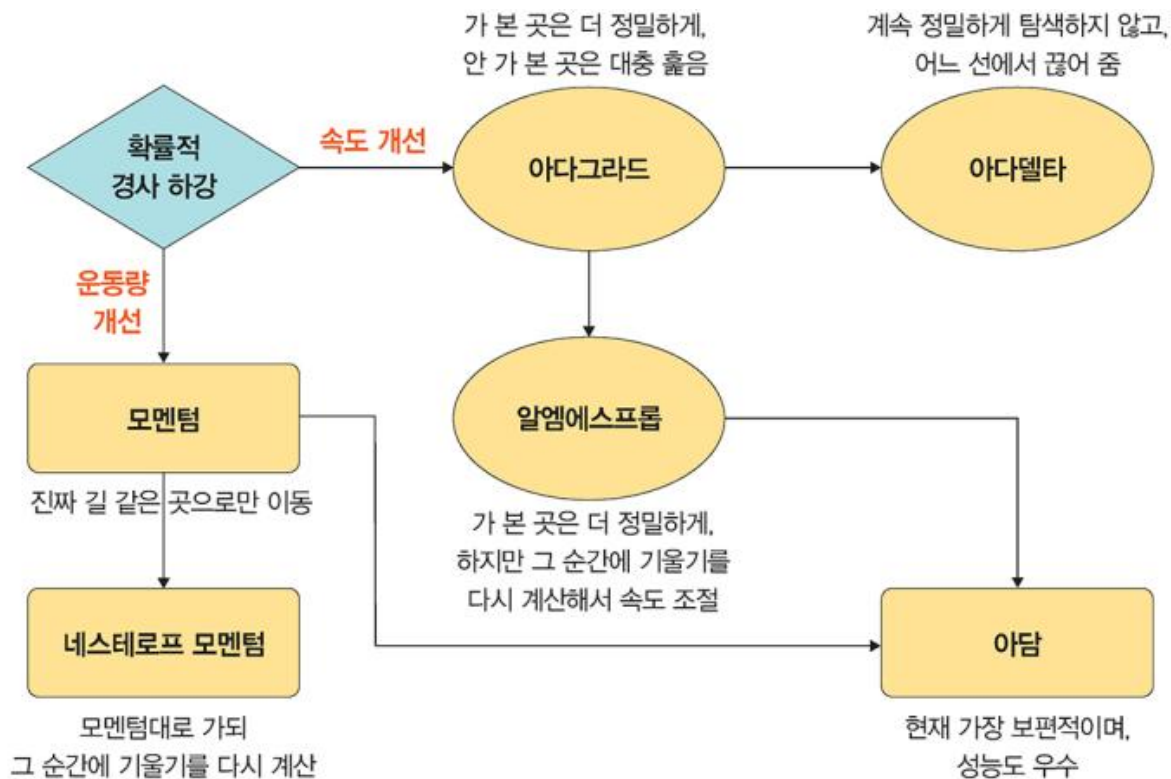
확률적 경사 하강법



미니 배치 경사 하강법

2. 딥러닝 구조 - 옵티마이저

- 옵티마이저 : 확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결
 - 학습속도와 운동량을 조절



2. 딥러닝 구조 - 옵티마이저(속도 조정방법)

- 아다그라드

- 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정
- 많이 변화한 변수 → 학습률 감소
- 적게 변화한 변수 → 학습률 증가
- 변수별로 다른 학습 속도를 적용하여 효율적인 학습 가능

- 아다델타

- 아다그라드의 학습 정지 문제를 개선 한 방법
- 누적된 변화량을 이용하여 학습률 자동 조정
- 별도의 학습률 설정이 필요 없음

- RMSProp

- 아다그라드의 학습률 감소 문제 개선
- 기울기 값을 일정 범위로 유지하여 안정적인 학습 가능

2. 딥러닝 구조 - 옵티마이저(운동량 개선)

- 모멘텀
 - 이전 기울기 방향을 반영하여 업데이트
 - 진동 감소 및 학습 속도 향상
 - 관성을 이용한 학습 방식
- 네스테로프 모멘텀
 - 이동 후 기울기를 다시 계산하여 방향 보정
 - 모멘텀보다 더 정확한 업데이트 가능
- Adam
 - 모멘텀 + RMSProp 결합
 - 학습 속도와 안정성 모두 향상
 - 가장 널리 사용되는 옵티마이저

2. 딥러닝 구조 - MLP로 MNIST 데이터셋 분류



https://colab.research.google.com/drive/1a0KofpMWMztaIGUyS_MZ8tpJrp7MwlCO?usp=sharing

2. 딥러닝 구조 - MLP로 MNIST 데이터셋 분류

구글코랩 → 런타임유형 변경 → GPU선택

런타임 유형 변경

런타임 유형

Python 3

하드웨어 가속기 ?

☐ CPU ☒ T4 GPU ☐ H100 GPU ☐ G4 GPU

☐ A100 GPU ☐ L4 GPU ☐ v5e-1 TPU

☐ v6e-1 TPU

i 프리미엄 GPU를 이용하시겠습니까? [추가 컴퓨팅 단위 구매](#)

런타임 버전 ?

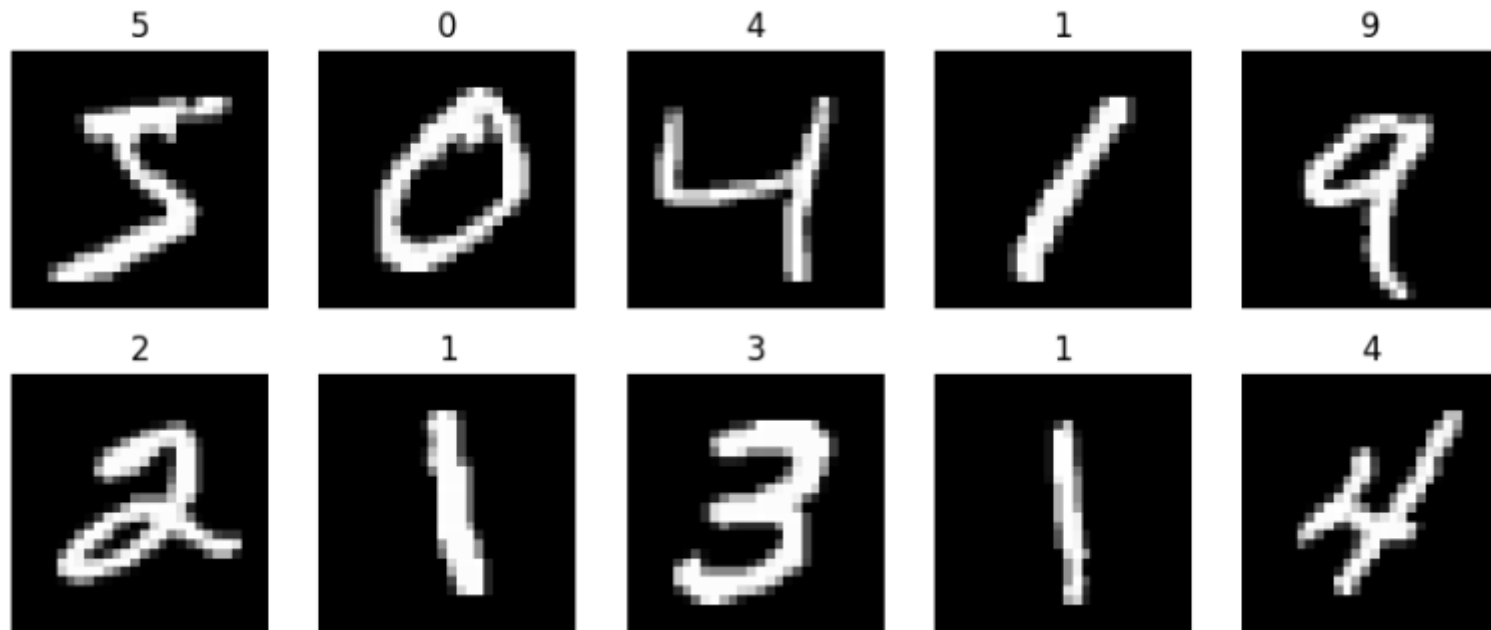
최신 버전(권장)

취소 저장

MNIST 데이터 셋을 불러와서,
MLP로 학습 한 뒤 숫자를 분류하는 것

라이브러리 불러오기
→ GPU 설정
→ 데이터 준비
→ 모델 만들기
→ 손실함수/최적화 설정
→ 학습
→ 성능 평가

2. 딥러닝 구조 - MLP로 MNIST 데이터셋 분류



Thank You